EXPERIMENT NO. 4

To Deploy Multi-Service Applications Using Docker Compose

1. Aim

To deploy a multi-service application consisting of a Python Flask web application and a MySQL database using Docker Compose.

2. Objectives

1. Understand Docker Compose and multi-service applications.
2. Create a Flask web application.
3. Configure MySQL as a separate service.
4. Create a Dockerfile for the Flask application.
5. Create docker-compose.yml to define multiple services.
6. Build and start multiple containers together.
7. Connect Flask with MySQL using the Compose service name.
8. Check service status and logs.
9. Stop and restart the multi-service application.

3. Requirements

- Windows OS
- Docker Desktop
- Visual Studio Code
- Git Bash / Command Prompt / PowerShell
- Web Browser
- Internet connection

4. Theory

4.1 Multi-Service Application

A multi-service application consists of multiple services that work together. In this experiment, the Flask web application and MySQL database run as separate Docker containers.

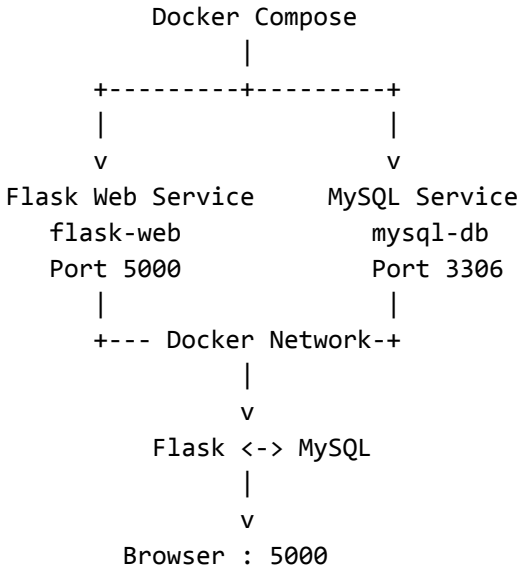
4.2 Docker Compose

Docker Compose is used to define and run multi-container applications. Services, images, ports, environment variables, and dependencies are specified in docker-compose.yml.

4.3 Services Used

Two services are used: web for the Flask application and db for MySQL.

5. Architecture



6. Project Structure

```
docker-compose-demo/
|
|-- app.py
|-- requirements.txt
|-- Dockerfile
`-- docker-compose.yml
```

7. Procedure

Step 1: Create Project Folder

Create a new folder named docker-compose-demo and open it in Visual Studio Code.

Step 2: Create app.py

Create app.py and enter:

```
from flask import Flask
import mysql.connector
```

```
app = Flask(__name__)

@app.route("/")
def home():
    try:
        connection = mysql.connector.connect(
            host="db",
            user="root",
            password="rootpassword",
            database="testdb"
        )

        if connection.is_connected():
            return "Flask application is running and connected to MySQL using
Docker Compose!"

    except Exception as e:
        return "Database connection failed: " + str(e)

    finally:
        if 'connection' in locals() and connection.is_connected():
            connection.close()

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create requirements.txt

Create requirements.txt and add:

```
Flask
mysql-connector-python
```

Step 4: Create Dockerfile

Create a file named Dockerfile without an extension:

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Step 5: Create docker-compose.yml

Create docker-compose.yml and add:

```
services:

  web:
    build: .
    container_name: flask-web
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: mysql:8.0
    container_name: mysql-db
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: testdb
    ports:
      - "3307:3306"
```

8. Explanation of docker-compose.yml

8.1 web service

The web service represents the Flask application. It is built from the Dockerfile, is named flask-web, and maps host port 5000 to container port 5000.

8.2 db service

The db service uses the mysql:8.0 image and is named mysql-db. It creates the testdb database and sets rootpassword as the root password.

8.3 depends_on

depends_on specifies that the web service depends on the db service.

8.4 Service Communication

The Flask application connects to MySQL using host name db. The name db is the Docker Compose service name and can be resolved between services on the Compose network.

8.5 Port Mapping

```
Flask: 5000:5000
MySQL: 3307:3306
```

Host port 3307 is used for MySQL to avoid conflicts with existing MySQL containers on the computer. Inside the Compose network, Flask communicates with MySQL on db:3306.

9. Build and Start All Services

Open Git Bash/PowerShell in the docker-compose-demo directory and run:

```
docker compose up --build
```

Docker Compose builds the Flask image, pulls the MySQL image if necessary, creates a network, creates both containers, and starts the services.

10. Check Running Containers

```
docker ps
```

The output should show flask-web and mysql-db as running. Flask should show 0.0.0.0:5000->5000/tcp and MySQL should show 0.0.0.0:3307->3306/tcp.

11. Check Compose Services

```
docker compose ps
```

This displays the services and their current status.

12. Access the Application

Open a browser and enter:

```
http://localhost:5000
```

Expected output:

```
Flask application is running and connected to MySQL using Docker Compose!
```

13. Check Flask Logs

```
docker compose logs web
```

14. Check MySQL Logs

```
docker compose logs db
```

15. Check All Service Logs

```
docker compose logs
```

16. Run Services in Background

```
docker compose up -d --build
```

The services now run in the background. Verify them with docker compose ps.

17. Stop and Remove Services

```
docker compose down
```

This stops and removes the Compose containers and network.

18. Start the Services Again

```
docker compose up -d
docker compose ps
```

Both services should start again. Open http://localhost:5000 to verify the application.

19. Important Docker Compose Commands

Command	Purpose
docker compose up --build	Build and start all services
docker compose up -d --build	Build and start services in background
docker compose up -d	Start services in background
docker compose ps	Display Compose service status
docker compose logs	Display logs of all services
docker compose logs web	Display Flask logs
docker compose logs db	Display MySQL logs
docker compose down	Stop and remove Compose containers and network
docker ps	Display running Docker containers

20. Observation

Service	Container Name	Port Mapping
Flask Web	flask-web	5000:5000
MySQL Database	mysql-db	3307:3306

21. Result

The multi-service application was successfully deployed using Docker Compose. The Flask web application and MySQL database were deployed as separate containers, and the Flask service successfully communicated with the MySQL service using the Docker Compose network.

22. Conclusion

The experiment demonstrated how Docker Compose can be used to define and deploy a multi-service application. A Flask web service and a MySQL database service were configured in docker-compose.yml, started together, verified using Compose commands, and accessed through the browser.

23. Viva Questions and Answers

Q: What is Docker Compose?

A: A tool used to define and run multi-container applications using a YAML configuration file.

Q: What is a multi-service application?

A: An application made up of multiple services that work together, such as a web application and database.

Q: What file is used by Docker Compose?

A: docker-compose.yml.

Q: How many services are used in this experiment?

A: Two: web for Flask and db for MySQL.

Q: Why are Flask and MySQL in separate containers?

A: For isolation and independent management of each service.

Q: What is the purpose of depends_on?

A: It specifies that the web service depends on the db service.

Q: Why does Flask use host='db'?

A: db is the Compose service name for MySQL and is used for service-to-service communication.

Q: What does docker compose up do?

A: It creates and starts the services defined in docker-compose.yml.

Q: What does docker compose down do?

A: It stops and removes the Compose containers and network.

Q: What does docker compose ps do?

A: It displays the status of the Compose services.

Q: Why is port 3307 used instead of 3306 on the host?

A: To avoid conflict with existing MySQL services using host port 3306.

Q: What does 3307:3306 mean?

A: 3307 is the host port and 3306 is the MySQL container port.

Q: How does Flask communicate with MySQL?

A: Through the Compose network using the service name db.

Q: What does -d mean?

A: It runs services in detached/background mode.

24. Final Practical Flow

```
docker-compose.yml
        |
   +----+----+
   |         |
   v         v
  web        db
   |         |
 Flask      MySQL
   |         |
   +----+----+
        |
 Docker Network
        |
   Flask <-> MySQL
        |
 Browser: http://localhost:5000
```